Task:6 MongoDB aggregation pipeline solutions

1. Given a MongoDB collection 'orders' with documents containing fields: userId, items (array of {product, quantity, price}), and orderDate, use the $match stage to find all orders placed in the last 30 days.

Ans: db.orders.aggregate([

```
  {
    $match: {
      orderDate: {
        $gte: new Date(
          new Date().setDate(new Date().getDate() - 30)
        )
```

```
    }
  }
 }
]);
```

2. Using the 'orders' collection, write an aggregation pipeline that groups orders by userId and calculates the total amount spent by each user using the $group stage

   Ans:
```
db.orders.aggregate([
 {
  $unwind: "$items"
 },
 {
```

```
    $group: {
      _id: "$userId",
      totalSpent: {
        $sum: {
          $multiply:
["$items.quantity", "$items.price"]
        }
      }
    }
  }
]);
```

3. Create an aggregation pipeline on a 'restaurants' collection (fields: name, cuisine, city, rating) to find the top 3 highest-rated cafes in Ahmedabad. Use $match for

cuisine, $sort for rating, and $limit for top results.

Ans: db.restaurants.aggregate([

```
  {
    $match: {
      cuisine: "Cafe",
      city: "Ahmedabad"
    }
  },
  {
    $sort: {
      rating: -1
    }
  },
  {
```

```
    $limit: 3
  }
]);
```

- $match filters cafes in Ahmedabad.
- $sort orders by rating in descending order.
- $limit returns only the top 3 cafes.
-

4. In a 'movies' collection (fields: title, genre, boxOffice), use $group to calculate the total box office collection for each genre and then use $project to display only genre and totalCollection fields in the output

   Ans: db.movies.aggregate([

```
  {
    $group: {
      _id: "$genre",
      totalCollection: {
        $sum: "$boxOffice"
      }
    }
  },
  {
    $project: {
      _id: 0,
      genre: "$_id",
      totalCollection: 1
    }
  }
]);
```

$group groups movies by genre.

$sum calculates total box office revenue per genre.
$project renames _id to genre and removes the default _id.

5. Use ChatGPT or GitHub Copilot to generate an aggregation pipeline that produces a daily sales report from a 'payments' collection (fields: userId, amount, paymentDate), showing total sales per day for the last week. Paste the generated code and briefly describe any changes you made to fit your collection's structure.

   Ans: db.payments.aggregate([
   {

```
    $match: {
      paymentDate: {
        $gte: new Date(
          new Date().setDate(new
Date().getDate() - 7)
        )
      }
    }
  },
  {
    $group: {
      _id: {
        $dateToString: {
          format: "%Y-%m-%d",
          date: "$paymentDate"
        }
      },
      totalSales: {
```

```
      $sum: "$amount"
    }
  }
 },
 {
  $sort: {
    _id: 1
  }
 },
 {
  $project: {
    _id: 0,
    date: "$_id",
    totalSales: 1
  }
 }
]);
```